How to build a Foundation Model for Univariate Time Series Forecasting: Practical Survey

Samy-Melwan Vilhes[1]

[1]INSA Rouen, Univ Rouen, Normandie Univ, LITIS UR 4108, F-76000 Rouen, France

September 2025

Abstract

In recent years, the success of large language models (LLMs) in text generation has inspired analogous approaches in time series forecasting. Large-scale models have been designed to take a signal as input and generate its future values. Rather than being limited to one signal type, these models are trained to generalize across diverse and even unseen signals, a property referred to as zero-shot forecasting. Moreover, they can be pretrained using transformer-based architectures and training strategies similar to those employed in LLMs. In this work, we propose a tutorial on how to train such a model, inspired by the training paradigm of LLMs. While LLMs rely on next-token prediction, our model is specifically designed for next-patch prediction in time series. Our goal is not to propose an entirely new approach, but to make both the model and its pretraining process transparent, accessible, and easy to understand, while avoiding the need for massive computational resources. By drawing on established methods and thoughtfully combining their components, we deliver a model and pretraining process that are both practical and easy to reproduce.
The code is available at: https://github.com/vilhess/PatchFM

1 Introduction

1.1 Motivation

Time series analysis supports applications across multiple domains such as finance, energy, manufacturing and healthcare. With the proliferation of IoT devices, data are collected at very high frequencies and at massive scale. Common tasks include anomaly detection for monitoring systems [23, 26], signal classification for early diagnosis [11], and, our focus, forecasting [16, 4]: given a context signal, predict future observations. Such forecasts are central in settings like weather and electricity load forecasting, where accuracy matters for decision making. Conventional forecasting pipelines often train a separate model per signals, achieving strong in-domain accuracy but poor transfer to other signals.

Differences in scale, dynamics, sampling rate, seasonality, and noise induce distribution shift; a model fit to one signal rarely generalizes to another. Inspired by LLMs [8, 17, 13], which acquire broad capabilities from the simple next-token prediction objective (e.g., multilinguality and coding), researchers are pursuing analogous approaches for time series via future-value prediction [4, 20, 7, 12, 3, 6, 24]. The goal is to build a foundation model that, given a context signal, forecasts future values across heterogeneous domains, enabling zero-shot forecasting.

Here, we present a deliberately simple architecture that closely mirrors those used for LLMs. Our goal is not novelty but clarity: explain how to train such a model, draw explicit connections to LLM practices, and make the approach easy to understand and reproduce. We provide a straightforward, reproducible recipe built from scalable, LLM-inspired components while keeping compute modest. The resulting framework aims at a foundation model that, given a context signal, forecasts future values across heterogeneous domains (zero-shot), quantifies uncertainty, and runs efficiently in practice.

2 Background

2.1 Large Language Models pretraining

Large Language Models (LLMs) for text generation are deep neural networks trained to model the probability distribution of token sequences in natural language. Pretraining typically relies on either autoregressive next-token prediction [13, 8, 17] or masked-token reconstruction [9]. In this work, we focus on the autoregressive objective, where the model is trained to predict the next token given all previous ones.

Text is first tokenized into a discrete vocabulary using a tokenizer such as Byte-Pair Encoding (BPE). The model then learns to assign a high probability to the correct next token conditioned on the preceding context.

Modern LLMs are built from Transformer blocks [22], composed of multi-head self-attention and feed-forward layers, with residual connections and layer normalization. Training minimizes the cross-entropy loss over a massive corpus of tokens.

The pretraining corpus is typically extensive and carefully curated to ensure both quality and diversity. It includes text from multiple domains such as books, academic articles, web pages, and source code, often spanning multiple languages. After pretraining, the model is capable of generating coherent text, answering questions, translating between languages, and even writing code. In a subsequent phase, LLMs are often fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [18] to align their behavior with human preferences. Further reinforcement learning techniques can enhance reasoning abilities, for instance by working on mathematical problem-solving.

In this work, when working with time series, we focus exclusively on the pretraining phase and do not consider any fine-tuning procedure.

2.2 Related Works

Recent pretrained foundation models for time-series forecasting differ along three axes: tokenization (discrete tokens vs. continuous patches), backbone (encoder–decoder vs. decoder-only vs. recurrent), and output granularity (point-wise vs. patch-wise).
Chronos [3] is closest to LLM pretraining: it discretizes $\mathbb{R}$ via quantization to form a finite vocabulary and casts forecasting as next-token prediction over the finite vocabulary. It uses the T5 encoder–decoder transformer [19], originally designed for text, and autoregressively predicts the next quantized value.
TimesFM [7] adopts a decoder-only transformer with patching, treating each patch as a token. A single forward pass can estimate multiple values (the patch length). Unlike discretized vocabularies, patch tokens live in a continuous space rather than a fixed finite set.
TimeMoE [20] integrates sparse Mixture-of-Experts within transformer layers and uses point-wise embeddings—predicting one value at a time rather than an entire patch.
TiRex [4] also operates patch-wise but replaces the transformer with the xLSTM backbone [5], exploring recurrent architectures under a patching scheme.
Most of the above target univariate forecasting and can be applied to multivariate data by processing each variable as a separate stream.
In parallel, zero-shot multivariate models such as **Toto** [6] **Tiny Time Mixers** [10] and **MOIRAI** [24] directly model cross-variable dependencies.
Among these, **Toto**, **TiRex**, **MOIRAI** and **Chronos** explicitly report uncertainty (e.g., quantiles/intervals) in the zero-shot setting.

2.3 Notations

We introduce here the notations that will be used throughout this work. These notations may be redefined locally when introduced in specific contexts. We denote:

- w: the size of the context window,
- $x_t \in \mathbb{R}$: the value of the signal at time t,
- $x_{t-w+1:t} \in \mathbb{R}^w$: the signal values from time $t-w+1$ to t,
- $x_{\text{ctx}} \in \mathbb{R}^w$: the context signal,
- P_{len}: number of observations in a patch (must be a divisor of w) the equivalent of the token embedding dimension in LLMs,
- $\mathrm{P}_{\text{num}} = \frac{w}{\mathrm{P}_{\text{len}}}$: the number of patches constructed from x_{ctx} ,
- $x_p \in \mathbb{R}^{\mathrm{P}_{\text{num}} \times \mathrm{P}_{\text{len}}}$: the patchified representation of x_{ctx},
- $x_{p_i} \in \mathbb{R}^{\mathrm{P}_{\text{len}}}$: the i-th patch of x_p,

- $x_{p_i,t}$: the t-th element of x_{p_i}
- $x_{\text{tar}} \in \mathbb{R}^{\mathrm{P}_{\text{len}}}$: the target patch, consisting of the P_{len} observations immediately following the context x_{ctx},
- $\mathcal{Q}$: the set of quantile levels,
- $\hat{x}_{p_i} \in \mathbb{R}^{|\mathcal{Q}| \times \mathrm{P}_{\text{len}}}$: stacked quantile predictions for patch i (row q is $\hat{x}_{p_i}^{(q)}$),
- $\hat{x}_{p_i}^{(q)} \in \mathbb{R}^{\mathrm{P}_{\text{len}}}$: the quantile-$q$ prediction vector for patch i ($q \in \mathcal{Q}$),
- $\text{concat}(x_{p_1}, \ldots, x_{p_i}) \in \mathbb{R}^{i\mathrm{P}_{\text{num}}}$: the concatenation of the first i patches, i.e. recovering a sequence of length $i\mathrm{P}_{\text{len}}$.

2.4 Patching for Time Series

The concept of patching for time series was introduced in [16], where the authors proposed a patch-based transformer architecture for time series forecasting. This approach demonstrated strong performance and has since become a standard building block: most subsequent works in time series modeling such as forecasting and anomaly detection adopt a patch-based representation [25, 12, 26, 23] and even already for zero-shot forecasting [4, 6, 7, 20]. An interesting improvement with patches is that a univariate time series can now be decomposed into multiple patches, analogous to tokens in LLMs, enabling the effective application of transformer-based architectures [22] to time series data and the training strategies for building state-of-the-art LLMs. However, while the number of tokens in LLMs is finite and corresponds to words or subwords, the number of patches in time series is infinite and corresponds to segments of real-valued data.

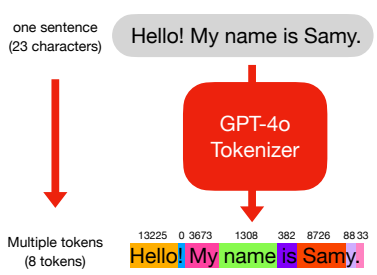


Figure 1: GPT-4o tokenizer

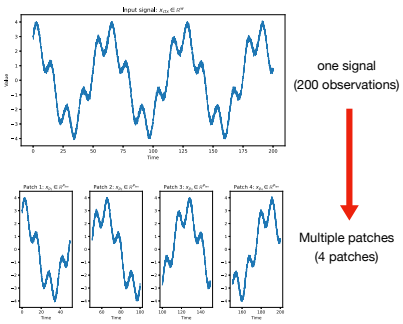


Figure 2: Time Series Patching

The input signal $x_{\text{ctx}} \in \mathbb{R}^w$ is divided into consecutive patches of length P_{len}. Thus, the number of patches is given by $\mathrm{P}_{\text{num}} = \frac{w}{\mathrm{P}_{\text{len}}}$. The patched representation of x_{ctx} is denoted by $x_p \in \mathbb{R}^{\mathrm{P}_{\text{num}} \times \mathrm{P}_{\text{len}}}$. We denote the i-th patch of x_p as x_{p_i}. A patch is the equivalent of a token when working with LLMs. A key difference

from LLMs is that while language models operate with a finite vocabulary of tokens, our time series patching strategy generates patches from a continuous space. Since time series values can take any real number, the space of possible patches is infinite, unlike the discrete token vocabulary in natural language processing. The number of patches in the context equals $\mathrm{P}_{\mathrm{num}}$. Furthermore, in LLMs, each token is mapped to a learned embedding vector in a high-dimensional space. In contrast, our patching strategy directly provides the vector: each patch is a length-$\mathrm{P}_{\mathrm{len}}$ vector, without requiring an embedding lookup table.

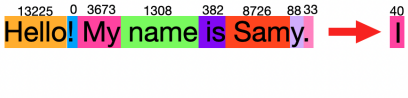


Figure 3: Next token prediction

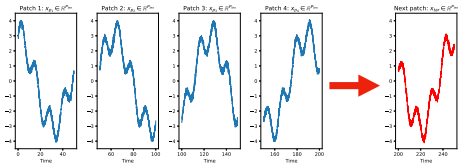


Figure 4: Next patch prediction

In LLMs for text generation, given the preceding tokens, the model selects the most probable next token via a classification strategy over the vocabulary. In our setting, given the context signal x_{ctx}, and specifically its patched representation, the model forecasts the next patch x_{tar}, a continuous vector of length $\mathrm{P}_{\mathrm{len}}$. During training, given the full sequence of patches, we learn an autoregressive predictor: for each position i, the model predicts the next patch $x_{p_{i+1}}$ using only $\{x_{p_1}, \ldots, x_{p_i}\}$. Thus, for a sequence of $n+1$ patches, we obtain n next-patch targets and can compute all n predictions (and their losses) in a single forward pass.

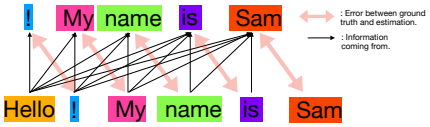


Figure 5: Training forward pass with tokens (LLMs).

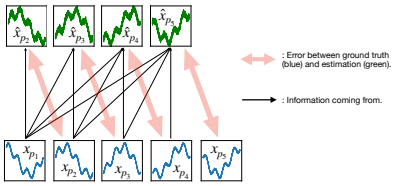


Figure 6: Training forward pass with patches (Forecasting).

3 Method

3.1 Problem Formulation

Let a univariate signal be $x_{1:T}$. For training, we extract a window of length w and set a patch length $\mathrm{P}_{\mathrm{len}}$ such that $\mathrm{P}_{\mathrm{num}} = w/\mathrm{P}_{\mathrm{len}} \in \mathbb{N}$. The context window

$x_{\text{ctx}} \in \mathbb{R}^w$ is split into consecutive patches

$$x_p = \left[x_{p_1}, \ldots, x_{p_{\mathrm{P}_{\text{num}}}}\right] \in \mathbb{R}^{\mathrm{P}_{\text{num}} \times \mathrm{P}_{\text{len}}}, \quad x_{p_i} \in \mathbb{R}^{\mathrm{P}_{\text{len}}}.$$

Next-patch prediction: for each position i, predict the following patch $x_{p_{i+1}}$ using only past patches $\{x_{p_1}, \ldots, x_{p_i}\}$. The model f_θ outputs quantile forecasts for the next patch:

$$f_\theta\big(x_{p_1}, \ldots, x_{p_i}\big) = \big\{\hat{x}^{(q)}_{p_{i+1}} \in \mathbb{R}^{\mathrm{P}_{\text{len}}} \ : \ q \in \mathcal{Q}\big\}.$$

Training uses causal masking so that attention from position i can only attend to indices $\leq i$. Given a sequence of $n{+}1$ consecutive patches $\{x_{p_1}, \ldots, x_{p_{n+1}}\}$, a single forward pass produces $\{\hat{x}^{(q)}_{p_{i+1}}\}_{i=1}^{n}$ in parallel, and the loss aggregates the pinball loss loss over positions, elements, and quantiles (see Multi-Quantile Loss).
At inference, the median (q=0.5) serves as the point forecast for the next patch; longer horizons are obtained by autoregressive rollout with KV caching.

3.2 Reversible Instance Normalization

As discussed in 2.4, the space of possible patches is effectively infinite, and their values can span very different scales. For example, one series may fluctuate around -0.03 with variations on the order of 10^{-4}, whereas another may be centered near 4×10^6 with variations on the order of 10^5. Such scale disparities can destabilize training and degrade performance. We therefore adopt Reversible Instance Normalization (RevIN) [14] and adapt it to our causal, patch-based training strategy.
RevIN is a normalization–denormalization method that first normalizes input sequences, then gives the new representation to the model, and finally denormalizes the model's outputs to address distribution shifts in time-series forecasting. RevIN is symmetrically structured to restore the original distribution information in the model output by scaling and shifting the outputs in the denormalization layer by the same amounts used for shifting and scaling the inputs during normalization. This technique has been adopted in most recent works on time series forecasting [16, 23, 26, 4, 6]. We propose to adopt this normalization technique when working with patches under a causal structure; that is, the normalization procedure for patch p_i does not use information from subsequent patches, but only from previous ones. This approach is also considered in [6], but not in [4], which are among the most recent zero-shot forecasting models. Furthermore, we apply the $\tanh^{-1}$ function to the normalized predictions to ensure handle extreme values such as in [2]. Mathematically, the normalization of each patch can be expressed as

$$x^{\text{norm}}_{p_i} = \tanh^{-1}\left(\frac{x_{p_i} - \mu_i}{\sqrt{\sigma_i^2}}\right),$$

with

$$\mu_i = \operatorname{mean}\big(\operatorname{concat}(x_{p_1}, \ldots, x_{p_i})\big), \quad \sigma_i^2 = \operatorname{var}\big(\operatorname{concat}(x_{p_1}, \ldots, x_{p_i})\big) + \epsilon,$$

where $\epsilon > 0$ ensures numerical stability. All operations are applied element-wise. Given the model's normalized prediction for the next patch, $\hat{x}^{\text{norm}}_{p_{i+1}}$, we invert the transform as

$$\underbrace{\hat{x}_{p_{i+1}}}_{\in \mathbb{R}^{\mathrm{P_{len}}}} = \underbrace{\tanh\left(\hat{x}^{\text{norm}}_{p_{i+1}}\right)}_{\in \mathbb{R}^{\mathrm{P_{len}}}} \odot \underbrace{\sqrt{\sigma_i^2}}_{\in \mathbb{R}} + \underbrace{\mu_i}_{\in \mathbb{R}}.$$

3.3 Architecture

The input signal $x_{\text{ctx}} \in \mathbb{R}^w$ is first processed into multiple patches $x_p \in \mathbb{R}^{\mathrm{P_{num}} \times \mathrm{P_{len}}}$. These patches are then normalized using RevIN. The model ingests these normalized patches and processes them with a causal Transformer to produce next-patch predictions (one per quantile). Patches are embedded, enriched with positional information (RoPE), and passed through stacked multi-head self-attention and SwiGLU feed-forward blocks with residual connections. The model provides predictions but on the normalized space. The resulting predictions are then denormalized using the corresponding μ_i and σ_i^2 to yield forecasts on the original scale.

3.3.1 Input and Output Projections

Each patch $x_{p_i} \in \mathbb{R}^{\mathrm{P_{len}}}$ is mapped to the model dimension dim by a lightweight 2-layer residual MLP with activation. After the stacked attentions and feed-forward blocks, the patch embedding is projected back to the patch space by $|\mathcal{Q}|$ parallel similar residual networks (one per quantile level): Stacking these heads yields $\hat{x}_{p_{i+1}} \in \mathbb{R}^{|\mathcal{Q}| \times \mathrm{P_{len}}}$.

3.3.2 Rotary Position Embeddings (RoPE)

To encode temporal order at the patch level, we use Rotary Position Embeddings (RoPE) [21]. RoPE applies a position-dependent rotation to query/key vectors in 2D subspaces, making the attention score

$$S_{i,j} = \left\langle \mathrm{RoPE}(q_i, i),\, \mathrm{RoPE}(k_j, j) \right\rangle$$

depends on the relative offset $(i - j)$ and not the absolute position in the sequence. This provides flexible sequence lengths without learned position tables, distance-aware decay of interactions, and clean integration with causal self-attention and KV caching. RoPE is widely used in modern Transformer architectures, e.g., LLaMA, DeepSeek, and GPT-family models [13, 8, 17]. Thus the RoPE transformation is applied to the query and key vectors before computing attention scores of every attention layer.

3.4 Causal Masking and Attention

Our task is next-patch prediction, which requires enforcing causality in the attention mechanism. Each patch may only attend to its past patches, never

to future ones. Allowing access to future information would cause data leakage and make real-time forecasting infeasible.
To achieve this, we apply causal masking. When computing the new representation of p_i, the attention weights between patch p_i and patch p_j are set to zero whenever $j > i$. This produces a lower-triangular attention map that preserves temporal order and ensures predictive consistency.
Causal masking is a standard technique in transformer-based architectures, commonly used in both language modeling and time series forecasting.

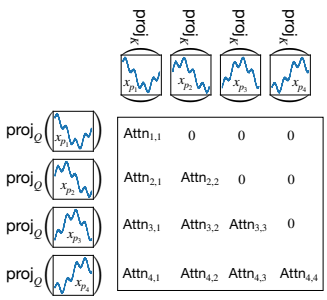


Figure 7: Attention matrix with causal masking.

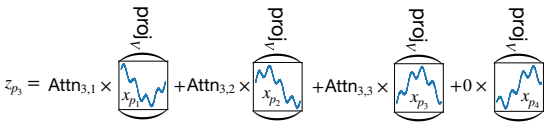


Figure 8: Output values derived from attention.

Each embedded patches of size dim from the input projection (3.3.1) is mapped into query, key, and value spaces through linear projections $\mathrm{proj}_Q, \mathrm{proj}_K, \mathrm{proj}_V \in \mathbb{R}^{\mathrm{dim}\times\mathrm{dim}}$. For simplicity, Figs. 7 and 8 depict the higher-dimensional patch (after the input projection) representations directly as x_{p_i}.
Attention between patches i and j is:

$$\mathrm{Attn}_{i,j} = \frac{\exp(S_{i,j})}{\sum_{k=1}^{i}\exp(S_{i,k})}, \quad S_{i,j} = \frac{\mathrm{RoPE}(\mathrm{proj}_Q(x_{p_i}), i)^\top \mathrm{RoPE}(\mathrm{proj}_K(x_{p_j}), j)}{\sqrt{\mathrm{dim}}}.$$

The updated representation of patch i is:

$$z_{p_i} = \sum_{j=1}^{i} \mathrm{Attn}_{i,j} \cdot \mathrm{proj}_V(x_{p_j}).$$

Therefore, after the attention layer, each patch representation is updated using only its predecessors' representations.

3.4.1 SwiGLU Feed-Forward

We use SwiGLU (SiLU-gated) feed-forward networks in each Transformer block. Given hidden state $h \in \mathbb{R}^{\mathrm{dim}}$,

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(h) = \big(\underbrace{hW_u}_{\in \mathbb{R}^{d_{\mathrm{ff}}}} \odot \underbrace{\mathrm{SiLU}(hW_v)}_{\in \mathbb{R}^{d_{\mathrm{ff}}}}\big)W_o,$$

where $W_u, W_v \in \mathbb{R}^{\mathrm{dim}\times d_{\mathrm{ff}}}$, $W_o \in \mathbb{R}^{d_{\mathrm{ff}}\times \mathrm{dim}}$, and $\mathrm{SiLU}(z) = z\,\sigma(z)$ with $\sigma(z) = 1/(1+e^{-z})$ and $d_{\mathrm{ff}} \approx \frac{2}{3}\cdot 4\,\mathrm{dim}$.

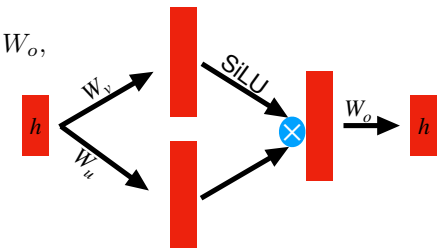


Figure 9: SwiGLU Feed Forward.

3.5 Training Objective - Multi-Quantile (Pinball) Loss

The model is trained not only to forecast the next patch but also to estimate predictive quantiles. We output quantiles $\mathcal{Q} = \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$, where the median ($q$=0.5) serves as the point forecast, and the remaining quantiles provide uncertainty bands.
During training, the model takes as input a sequence of n patches $x_{p_1}, \ldots, x_{p_n}$. For each position $i \in \{1, \ldots, n\}$, it outputs quantile predictions for the next patch $x_{p_{i+1}}$, which is conditioned on the past patches $x_{p_1}, \ldots, x_{p_i}$. We denote the prediction at quantile q by $\hat{x}^{(q)}_{p_{i+1}} \in \mathbb{R}^{\mathrm{P_{len}}}$.
Then, the training objective is the multi-quantile (pinball loss) loss. Aggregated over positions, patch elements, and quantiles, it is defined as:

$$\mathcal{L} = \frac{1}{n\,\mathrm{P_{len}}\,|\mathcal{Q}|} \sum_{i=1}^{n} \sum_{t=1}^{\mathrm{P_{len}}} \sum_{q\in\mathcal{Q}} \rho_q\Big(x_{p_{i+1},t} - \hat{x}^{(q)}_{p_{i+1},t}\Big)\,.$$

where the pinball loss loss ρ_q is

$$\rho_q(u) = \begin{cases} q\,u, & u \geq 0, \\ (q-1)\,u, & u < 0. \end{cases}$$

.

3.6 Model details

Table 1: Model configuration

Patch size	32
Max context (patches)	32
Forecast horizon	32 (a single patch)
Quantiles $\mathcal{Q}$	$\{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$
Model dimension dim	2048
Number of layers	6
Attention heads per layer	64
Head dimension	32
Parameters	300M

4 Inference

4.1 Long-term Forecasting and KVcache

For clarity, we describe forecasting in terms of the median (q=0.5) point forecast; the procedure is identical when generating multiple quantiles.

During inference, a single forward pass predicts the next patch, i.e., the following $\mathrm{P}_{\mathrm{len}}$ values. To forecast longer horizons, we roll out autoregressively: append the predicted patch to the context (optionally dropping the oldest patch if the maximum number of context patches is reached) and repeat. After k steps, this yields $k \times \mathrm{P}_{\mathrm{len}}$ forecasted values.

To make rollout efficient, we use key–value (KV caching): at each step we reuse cached keys and values for past patches and compute them only for the newly added patch. This avoids recomputing attention over the full history at every step and reduces latency but increases memory usage.

Suppose the context contains c patches and we wish to forecast the next k patches (i.e., $k \times \mathrm{P}_{\mathrm{len}}$ values). In a naive rollout, to predict $\hat{x}_{p_{c+1}}$ we compute queries, keys, and values for all c context patches and form a $c \times c$ attention matrix in every head and layer. After appending $\hat{x}_{p_{c+1}}$ to the context, predicting $\hat{x}_{p_{c+2}}$ repeats the process for c+1 patches, and so on. The total cost over k steps scales as $\mathcal{O}\big(\sum_{s=0}^{k-1}(c+s)^2\big)$ for a single attention head.

With KV caching, we leverage causality: representations of past patches do not depend on future ones. Hence, per-layer, per-head keys and values for the existing context patches can be cached and reused. At the first step (predicting $\hat{x}_{p_{c+1}}$), we compute and cache keys/values for the c context patches. At the next step (predicting $\hat{x}_{p_{c+2}}$), we compute a new query and keys/values only for the newly appended patch $\hat{x}_{p_{c+1}}$, then compute attention scores against the cached history plus this new entry. This avoids recomputing attention over the full history and reduces latency, at the cost of storing keys and values for all past

patches. The total cost over k steps scales as $\mathcal{O}\big(c^2 + \sum_{s=1}^{k-1}(c+s)\big)$ for a single attention head.

Figures 10 and 11 illustrate predicting two future patches from four context patches using a two-layer Transformer with single-head attention. This requires two autoregressive rollout steps.

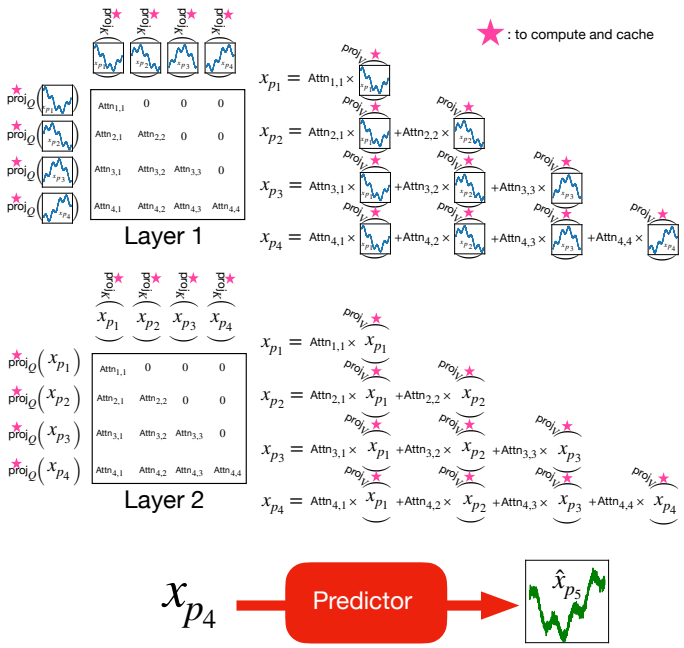


Figure 10: First rollout: KV caching.

Figure 10 shows the first rollout. For the four context patches, we compute the query, key, and value projections at each layer and cache the keys/values. This forward pass yields the prediction for the fifth patch.

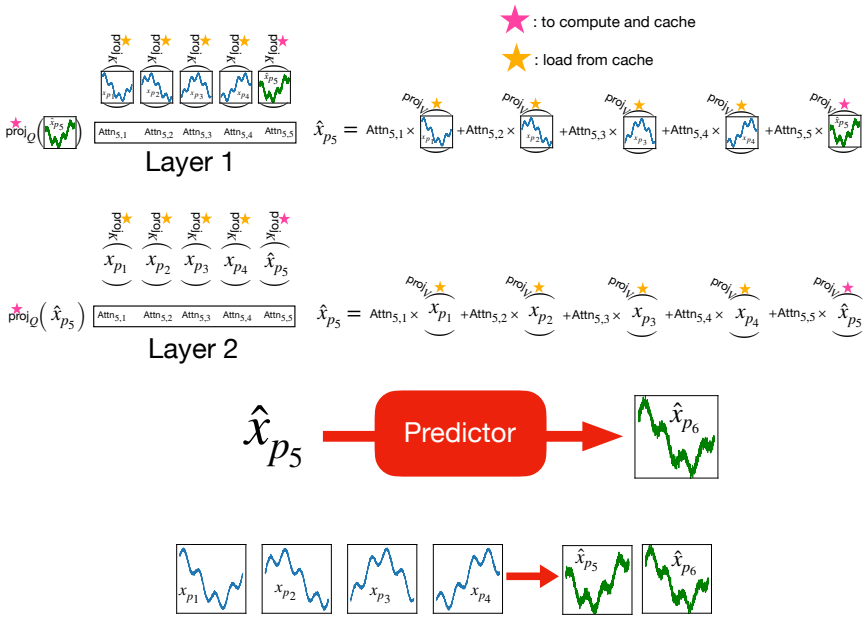


Figure 11: Second rollout: KV caching and reuse.

For the second rollout (Figure 11), we compute projections only for the newly predicted fifth patch and reuse the cached keys/values for all previous patches. Under causal masking, past representations do not depend on future patches, so no recomputation is needed. This step yields the sixth-patch prediction.

5 Training

5.1 Training Data

To train such a model, we need a large and diverse corpus of time series so it learns to generalize across signals rather than memorizing individual ones. We therefore combine synthetic datasets, covering varied scales, dynamics, sampling rates, seasonality, noise, and distribution shifts, with real-world data from industrial sensors, electricity load, and weather.

5.1.1 Artificial Dataset

We synthesize approximately 1M series from basic families such as sinusoidal, linear, polynomial, and logarithmic, and their mixtures, using TSMixup [3]

to increase pattern diversity. In addition, we adopt the KernelSynth strategy from [3] to sample Gaussian-process signals with mixtures of kernels (e.g., RBF, periodic, linear), sweeping hyperparameters (amplitudes, frequencies/periods, length-scales, noise) to cover a broad range of dynamics.

5.1.2 Real-World Dataset

For real-world signals, we leverage two datasets. The first one is the Unified Time Series Dataset (UTSD) [15], spanning seven domains: Energy, IoT, Nature, Web, Health, Transport, and Environment. UTSD is organized hierarchically to provide multiple benchmark subsets; each smaller dataset is a strict subset of a larger one, enabling scalable training and evaluation. It aggregates publicly available time series curated from diverse research groups and providers. For our pretraining, we use the biggest UTSD tier (UTSD-12G), which yields approximately 19M signals under our preprocessing pipeline. The second one is the GIFT-Eval pre-training datasets [1] aligned with the GIFT-Eval benchmark but without data leakage issue with the train/test split of the benchmark in order to assess fairly foundation models on GIFT-Eval. The dataset contains approximately 71 univariate and 17 multivariate time series datasets from various domains and various frequencies. After preprocessing, this yields approximately 1M univariate time series.

5.2 Setup

We train on 4 NVIDIA V100 GPUs using GENCI–IDRIS HPC resources. The implementation uses PyTorch Lightning with Distributed Data Parallel (DDP). Unless otherwise stated, we train for 5 epochs with a global batch size of 256.

5.2.1 What is DDP?

Distributed Data Parallel (DDP) replicates the model across multiple GPUs (one process per GPU). Each replica receives a different mini-batch, runs its own forward and backward passes, and then synchronizes gradients via an all-reduce. The averaged gradients are identical on every process, so the optimizer step updates all replicas to the same weights.
Concretely, per training step:

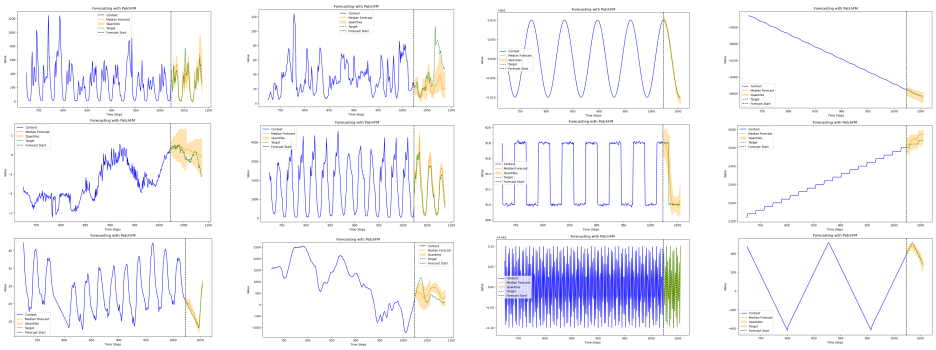
1. Each GPU loads a distinct shard of the batch.
2. Forward pass and local loss/backward are computed independently.
3. Gradients are all-reduced (averaged) across GPUs.
4. The optimizer step applies the same update on every replica.

This provides synchronous training with near-linear scaling and consistent parameters across GPUs.

6 Future works

- Improve performance: extend training duration.
- Benchmarking: evaluate on standard SOTA datasets with common metrics (e.g., MAE/MSE and quantile coverage) to compare against baselines.
- Ablations: gradually increase context length, RevIN (on/off and causal variants),
- Mixture of Experts: test sparse MoE in FFNs with routing,
- Implement LoRA for finetuning.

7 Some forecasting visualisations



(a) UTSD examples. (b) Artificial examples.

Figure 12: Forecasting visualisations considering the median with quantiles 0.1 and 0.9.

Glossary

context The input window (sequence of patches or raw time steps) used to condition the model's next-patch predictions. 1–5, 10, 11, 14

KV caching Key–Value caching: reuse of per-layer key/value tensors for past context during autoregressive rollout to avoid recomputation. 6, 7, 10–12

LLM Large Language Model: a model pretrained on large text corpora, typically via next-token or masked objectives. 1–4

MoE Mixture of Experts: architecture that routes tokens to a subset of specialized feed-forward experts to increase capacity efficiently. 14

patch A contiguous subsequence of a univariate time series (typically of fixed length P_{len}) used as the basic model unit; a continuous-valued vector analogous to a token in LLMs. 1, 4–12

patch length The number of time steps in a patch (denoted P_{len}). Equivalent to the dimension of a token embedding in LLMs. 3

pinball loss Quantile (pinball) loss used to train predictive quantiles; asymmetric linear loss controlled by quantile level q. 6, 9

RevIN Reversible Instance Normalization: per-instance normalization/denormalization to mitigate scale and distribution shifts in time series. 6, 7, 14

RoPE Rotary Position Embedding: a position encoding that rotates query/key subspaces to encode relative positions. 7

SwiGLU A gated feed-forward activation (SiLU gating) used in FFNs of Transformer blocks. 7–9

token A discrete subword unit produced by text tokenizers (e.g., BPE); tokens form a finite vocabulary used by LLMs. 1–5

UTSD Unified Time Series Dataset[15]: our corpus of signals from diverse domains for pretraining including 7 domains with up to 1 billion time points. 13, 14

zero-shot forecasting The ability of a model to forecast unseen signals without additional training, similar to how LLMs can generate text for prompts they have not explicitly been trained on. 1, 2, 4, 6

References

[1] Taha Aksu et al. *GIFT-Eval: A Benchmark For General Time Series Forecasting Model Evaluation*. 2024. arXiv: 2410.10393 [cs.LG]. URL: https://arxiv.org/abs/2410.10393.

[2] Abdul Fatir Ansari et al. *Chronos-2: From Univariate to Universal Forecasting*. 2025. arXiv: 2510.15821 [cs.LG]. URL: https://arxiv.org/abs/2510.15821.

[3] Abdul Fatir Ansari et al. *Chronos: Learning the Language of Time Series*. 2024. arXiv: 2403.07815 [cs.LG]. URL: https://arxiv.org/abs/2403.07815.

[4] Andreas Auer et al. *TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning*. 2025. arXiv: 2505.23719 [cs.LG]. URL: https://arxiv.org/abs/2505.23719.

[5] Maximilian Beck et al. *xLSTM: Extended Long Short-Term Memory*. 2024. arXiv: 2405.04517 [cs.LG]. URL: https://arxiv.org/abs/2405.04517.

[6] Ben Cohen et al. *This Time is Different: An Observability Perspective on Time Series Foundation Models*. 2025. arXiv: 2505.14766 [cs.LG]. URL: https://arxiv.org/abs/2505.14766.

[7] Abhimanyu Das et al. *A decoder-only foundation model for time-series forecasting*. 2024. arXiv: 2310.10688 [cs.CL]. URL: https://arxiv.org/abs/2310.10688.

[8] DeepSeek-AI et al. *DeepSeek-V3 Technical Report*. 2025. arXiv: 2412.19437 [cs.CL]. URL: https://arxiv.org/abs/2412.19437.

[9] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *CoRR* abs/1810.04805 (2018). arXiv: 1810.04805. URL: http://arxiv.org/abs/1810.04805.

[10] Vijay Ekambaram et al. *Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series*. 2024. arXiv: 2401.03955 [cs.LG]. URL: https://arxiv.org/abs/2401.03955.

[11] Hassan Ismail Fawaz et al. "InceptionTime: Finding AlexNet for Time Series Classification". In: *CoRR* abs/1909.04939 (2019). arXiv: 1909.04939. URL: http://arxiv.org/abs/1909.04939.

[12] Mononito Goswami et al. *MOMENT: A Family of Open Time-series Foundation Models*. 2024. arXiv: 2402.03885 [cs.LG]. URL: https://arxiv.org/abs/2402.03885.

[13] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[14] Taesung Kim et al. "Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift". In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=cGDAkQo1C0p.

[15] Yong Liu et al. "Timer: Generative Pre-trained Transformers Are Large Time Series Models". In: *Forty-first International Conference on Machine Learning*.

[16] Yuqi Nie et al. "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=Jbdc0vTOcol.

[17] OpenAI et al. *gpt-oss-120b and gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[18] Long Ouyang et al. *Training language models to follow instructions with human feedback*. 2022. arXiv: 2203.02155 [cs.CL]. URL: https://arxiv.org/abs/2203.02155.

[19] Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *CoRR* abs/1910.10683 (2019). arXiv: 1910.10683. URL: http://arxiv.org/abs/1910.10683.

[20] Xiaoming Shi et al. *Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts*. 2025. arXiv: 2409.16040 [cs.LG]. URL: https://arxiv.org/abs/2409.16040.

[21] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL]. URL: https://arxiv.org/abs/2104.09864.

[22] Ashish Vaswani et al. "Attention is all you need". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017. ISBN: 9781510860964.

[23] Samy-Melwan Vilhes, Gilles Gasso, and Mokhtar Z Alaya. *PatchTrAD: A Patch-Based Transformer focusing on Patch-Wise Reconstruction Error for Time Series Anomaly Detection*. 2025. arXiv: 2504.08827 [cs.LG]. URL: https://arxiv.org/abs/2504.08827.

[24] Gerald Woo et al. *Unified Training of Universal Time Series Forecasting Transformers*. 2024. arXiv: 2402.02592 [cs.LG]. URL: https://arxiv.org/abs/2402.02592.

[25] Yunhao Zhang and Junchi Yan. "Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=vSVLM2j9eie.

[26] Zhijie Zhong et al. *PatchAD: A Lightweight Patch-based MLP-Mixer for Time Series Anomaly Detection*. 2024. arXiv: 2401.09793 [cs.LG]. URL: https://arxiv.org/abs/2401.09793.